

TARGET BUSINESS CASE STUDY

1.Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1.1 Data type of all columns in the "customers" table.

Query:

```
select column_name, data_type
from
`scaler-dsml-sql-444610.target.INFORMATION_SCHEMA.COLUMNS`
where table_name='customers'
```

Output

```
60 select column_name, data_type
61 from `scaler-dsml-sql-444610.target.INFORMATION_SCHEMA.COLUMNS`
62 where table_name='customers'
```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DET
Row	column_name	data_type			
1	customer_id	STRING			
2	customer_unique_id	STRING			
3	customer_zip_code_prefix	INT64			
4	customer_city	STRING			
5	customer_state	STRING			

Insights

It is clear that the table 'Customers' has 5 Columns and out of which 4 are of data type String and 1 is of Integer type.

1.2. Get the time range between which the orders were placed.

I think the question is asked for the range, so my query is as given here. In this I have considered all the order_status. As question did not specify to not consider the order which are cancelled or not shipped.

Query1:

```
select
  MIN(order_purchase_timestamp) AS earliest_order_date,
  MAX(order_purchase_timestamp) AS latest_order_date
from `target.orders`
```

Output

Row	earliest_order_date	latest_order_date
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Insights

By looking at the output, one can easily make a note of the earliest_order date and latest_order_date. Based on this data one can count the duration period by days or months. Depending on the question.

If the question was asked to give the days/time between the orders placed, then my query is

Query2:

```
select timestamp_diff(latest_order_date, earliest_order_date,
Minute) as time_range_in_minutes
from
```

```
(select
  MIN(order_purchase_timestamp) AS earliest_order_date,
  MAX(order_purchase_timestamp) AS latest_order_date
from `target.orders`) tbl
```

Output

Row	time_range_in_minutes
1	1112894

Insights

By this output, we easily find the time range between which the orders are placed.

1.3. Count the Cities & States of customers who ordered during the given period.

Query:

```
SELECT
  COUNT(distinct c.customer_city) AS city_count,
  COUNT(distinct c.customer_state) AS state_count
FROM `target.orders` o
JOIN `target.customers` c ON o.customer_id = c.customer_id
WHERE
  order_purchase_timestamp BETWEEN (
    select MIN(order_purchase_timestamp) FROM
    `target.orders`)
```

```
AND (select MAX(order_purchase_timestamp) from
`target.orders`)
```

Output

JOB INFORMATION		RESULTS	CHART
Row	city_count ▼	state_count ▼	
1	4119	27	

Insights

Here, we get the count of cities as 4119 and state count as 27 and these are the count of unique or distinct city and states.

2. In-depth Exploration:

2.1 Is there a growing trend in the no. of orders placed over the past years?

Here, they have not mentioned, how the output should be displayed. So, I have considered to show the current year total orders as total_orders and previous year as prev_year_orders and I have made the difference of these 2 as order_difference and to calculate the percentage change, I have used the formulae,

$$(\text{total_orders} - \text{prev_year_orders}) / \text{prev_year_orders} * 100$$

And the question doesn't specify to show the increase percentage or what just growing or declining, So I have used a column to show the Trend is growing or not compared to previous year.

Query:

```
SELECT year, total_orders,
       LAG(total_orders) OVER(ORDER BY year) as prev_year_orders,
       total_orders - LAG(total_orders) OVER(ORDER BY year) AS
order_difference,
       ROUND((total_orders -LAG(total_orders) OVER(ORDER BY year))
/ LAG(total_orders)
          OVER(ORDER BY year) * 100, 2) AS percent_change,
CASE
  WHEN total_orders > LAG(total_orders) OVER(ORDER BY year)
  THEN 'Growing'
  WHEN total_orders < LAG(total_orders) OVER(ORDER BY year)
  THEN 'Declining'
  WHEN LAG(total_orders) OVER(ORDER BY year) IS NULL
  THEN 'No Previous Data'
  ELSE 'Stable'
END AS Trend
FROM
(SELECT
  extract(Year from order_purchase_timestamp) AS year,
  COUNT(*) AS total_orders
FROM `target.orders`
GROUP BY year) tbl
ORDER BY year
```

Output

Row	year	total_orders	prev_year_orders	order_difference	percent_change	Trend
1	2016	329	null	null	null	No Previous Data
2	2017	45101	329	44772	13608.51	Growing
3	2018	54011	45101	8910	19.76	Growing

Insights

One can easily observe that, By looking at the column percentage_change, It will tell us that there is an increase in growth per year. This indicates that there is good growth of the company.

2.2 Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Query:

```

SELECT
    EXTRACT(MONTH from order_purchase_timestamp ) as month_num,
    FORMAT_TIMESTAMP('%b', TIMESTAMP(order_purchase_timestamp)) AS
month_name,
    COUNT(order_id) AS total_orders
FROM
    `target.orders`
GROUP BY
    month_num, month_name
ORDER BY 1

```

Here, I have displayed month no (1 for Jan, 2 for Feb) and month name and order placed in that particular month as total_orders

Output

Untitled query RUN SAVE DOWNLOAD SHARE SCHEDULE OPEN IN MORE Query compl

```
123 -- EXTRACT(MONTH FROM order_purchase_timestamp) AS month_num,
124 -- FORMAT_TIMESTAMP('%b', TIMESTAMP(order_purchase_timestamp)) AS month_name,
125 -- COUNT(order_id) AS total_orders
126 FROM
127 -- 'target.orders'
128 GROUP BY
```

Query results SAVE RESULTS OPEN IN

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	month_num	month_name	total_orders			
1	1	Jan	8069			
2	2	Feb	8508			
3	3	Mar	9893			
4	4	Apr	9343			
5	5	May	10573			
6	6	Jun	9412			
7	7	Jul	10318			
8	8	Aug	10843			
9	9	Sep	4305			
10	10	Oct	4959			
11	11	Nov	7544			
12	12	Dec	5674			

Results per page: 50 1 - 12 of 12

Insights

From this output , we can see that Months like August has the highest no of orders and September has the lowest no of orders.

2.3 During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- ☐ 0-6 hrs : Dawn
- ☐ 7-12 hrs : Mornings
- ☐ 13-18 hrs : Afternoon
- ☐ 19-23 hrs : Night

Query:

```

SELECT
CASE WHEN Hour_Minute BETWEEN '00:00' and '06:59'
      THEN 'Dawn'
      WHEN Hour_Minute BETWEEN '07:00' and '12:59'
      THEN 'Mornings'
      WHEN Hour_Minute BETWEEN '13:00' and '18:59'
      THEN 'Afternoon'
      WHEN Hour_Minute BETWEEN '19:00' and '23:59'
      THEN 'Night'
END AS Time_Of_The_Day,
COUNT(tbl.order_id) as Order_Count
FROM
(SELECT order_id,
      FORMAT_TIMESTAMP ('%H:%M',TIMESTAMP(order_purchase_timestamp)) AS Hour_Minute
FROM `target.orders`) tbl
GROUP BY Time_Of_The_Day
ORDER BY Order_Count

```

Here, I have considered for eg: In the question its mentioned as 0-6 hrs it is dawn and 7-12 as mornings. But in my query, I have included 00:6.59 as 'Dawn'

Output

Row	Time_Of_The_Day ▼	Order_Count ▼
1	Dawn	5242
2	Mornings	27733
3	Night	28331
4	Afternoon	38135

Insights

From this we can come to the conclusion that Brazilian Customers most likely place their orders especially during Afternoon and rarely during the Dawn.

3. Evolution of E-commerce orders in the Brazil region:

3.1. Get the month on month no. of orders placed in each state.

Query:

```
SELECT c.customer_state,
       EXTRACT(MONTH FROM o.order_purchase_timestamp) as months,
       COUNT(order_id) as total_orders
FROM `target.orders` o JOIN `target.customers` c on o.customer_id=c.customer_id
GROUP BY 1,2
ORDER BY 1,2
```

Output

Row	customer_state	months	total_orders
1	AC	1	8
2	AC	2	6
3	AC	3	4
4	AC	4	9
5	AC	5	10
6	AC	6	7
7	AC	7	9
8	AC	8	7
9	AC	9	5
10	AC	10	6

Results per page: 50 1 – 50 of 322

Insights

From this output, we can check for the Customer state and the orders made per each month, based on this whole output, we can check the maximum orders made per state according to the month.

3.2. How are the customers distributed across all the states?

Query:

```
SELECT customer_state,  
       COUNT(DISTINCT customer_id) as Count_Of_Customers  
FROM `target.customers`  
GROUP BY customer_state  
ORDER BY 2 DESC, 1
```

Here, It displays the state with more no of customers first

Output

Row	customer_state	Count_Of_Customers
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

Results per page: 50 1 – 27 of 27

Insights

Customers are distributed across 27 states out of which State SP has the maximum no of customers.

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

4.1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

You can use the "payment_value" column in the payments table to get the cost of orders.

Query:

```
WITH year2017 AS (SELECT SUM(p.payment_value) as sum_2017,
                        ROW_NUMBER() OVER() as row1
                    FROM `target.orders` o join `target.payments` p on
o.order_id=p.order_id
                    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) =2017 AND
                        EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8 ),

year2018 AS (SELECT SUM(p.payment_value) as sum_2018,
                        ROW_NUMBER() OVER() as row1
                    FROM `target.orders` o join `target.payments` p on o.order_id=p.order_id
                    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) =2018 AND
                        EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8 )

SELECT ROUND((100*(b.sum_2018 - a.sum_2017 )/a.sum_2017 ), 2) AS Percentage_Increase
FROM    year2017 a JOIN year2018 b on a.row1=b.row1
```

Output

Untitled query

RUN
 SAVE
 DOWNLOAD
 SHARE
 SCHEDULE

```

279 WITH year2017 AS (SELECT SUM(p.payment_value) as sum_2017,
280                    ROW_NUMBER() OVER() as row1
281                    FROM `target.orders` o join `target.payments` p on o.order_id=p.order_id
282                    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) =2017 AND
283                          EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8 ),|
284
285   year2018 AS (SELECT SUM(p.payment_value) as sum_2018,
286                ROW_NUMBER() OVER() as row1
287                FROM `target.orders` o join `target.payments` p on o.order_id=p.order_id
288                WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) =2018 AND
289                      EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8 )
290
291 SELECT ROUND((100*(b.sum_2018 - a.sum_2017 )/a.sum_2017 ), 2) AS Percentage_Increase
292 FROM   year2017 a JOIN year2018 b on a.row1=b.row1
293

```

Query results

SAVE RESULTS

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	Percentage_Increase					
1	136.98					

Insights

We can observe that the percentage increase in the no of orders are 137% from 2017 to 2018.

4.2. Calculate the Total & Average value of order price for each state.

Query:

```

SELECT c.customer_state, ROUND(SUM(p.payment_value), 2) AS Total_Price,
      ROUND(AVG(p.payment_value), 2) AS Average_Price
FROM `target.payments` p JOIN `target.orders` o on p.order_id =o.order_id
      JOIN `target.customers` c on c.customer_id =o.customer_id
GROUP BY c.customer_state
ORDER BY c.customer_state

```

Output

Untitled query

RUN

OPEN IN

MORE

SAVE

DOWNLOAD

SHARE

SCHEDULE

```

297
298 SELECT c.customer_state, ROUND(SUM(p.payment_value), 2) AS Total_Price,
299        ROUND(AVG(p.payment_value), 2) AS Average_Price
300 FROM `target.payments` p JOIN `target.orders` o on p.order_id = o.order_id
301        JOIN `target.customers` c on c.customer_id = o.customer_id
302 GROUP BY c.customer_state
303 ORDER BY c.customer_state
304

```

Press Alt

Query results

SAVE RESULTS

JOB INFORMATION

RESULTS

CHART

JSON

EXECUTION DETAILS

EXECUTION GRAPH

Row	customer_state	Total_Price	Average_Price
1	AC	19680.62	234.29
2	AL	96962.06	227.08
3	AM	27966.93	181.6
4	AP	16262.8	232.33
5	BA	616645.82	170.82
6	CE	279464.03	199.9
7	DF	355141.08	161.13
8	ES	325967.55	154.71
9	GO	350092.31	165.76
10	MA	152523.02	198.86
11	NC	163007.66	164.75

Results per page:

50

1 – 27 of 27

Insights

In this output, we can see that Total price and Average price for each state.

4.3. Calculate the Total & Average value of order freight for each state.

Query:

```
SELECT c.customer_state, ROUND(SUM(oi.freight_value), 2) AS Total_Freight,
       ROUND(AVG(oi.freight_value), 2) AS Average_Freight
```

```

FROM `target.order_items` oi JOIN `target.orders` o on oi.order_id =o.order_id
      JOIN `target.customers` c on c.customer_id =o.customer_id
GROUP BY c.customer_state
ORDER BY c.customer_state

```

Output

Query results

Row	customer_state	Total_Freight	Average_Freight
1	AC	3686.75	40.07
2	AL	15914.59	35.84
3	AM	5478.89	33.21
4	AP	2788.5	34.01
5	BA	100156.68	26.36
6	CE	48351.59	32.71
7	DF	50625.5	21.04
8	ES	49764.6	22.06
9	GO	53114.98	22.77
10	MA	31523.77	38.26
11	MG	270853.46	20.63
12	MS	19144.03	23.37

Insights

This output tells the total freight and average freight value based on the states. We can sort the table according to our requirement.

5. Analysis based on sales, freight and delivery time.

5.1. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- $\text{time_to_deliver} = \text{order_delivered_customer_date} - \text{order_purchase_timestamp}$
- $\text{diff_estimated_delivery} = \text{order_delivered_customer_date} - \text{order_estimated_delivery_date}$

Query:

```
SELECT order_id,  
       DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS  
time_to_deliver,  
       DATE_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY)  
AS diff_estimated_delivery  
FROM `target.orders`  
WHERE order_delivered_customer_date IS NOT NULL  
ORDER BY 1
```

Output

336	
337	SELECT order_id,
338	DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS time_to_deliver,
339	DATE_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY) AS diff_estimated_delivery
340	FROM `target.orders`
341	WHERE order_delivered_customer_date IS NOT NULL
342	ORDER BY 1
343	

Query results	SAVE RESULTS
JOB INFORMATION	RESULTS
Row	order_id
1	00010242fe8c5a6d1ba2dd792...
2	00018f77f2f0320c557190d7a1...
3	000229ec398224ef6ca0657da...
4	00024acbcd0a6daa1e931b03...
5	00042b26cf59d7ce69dfabb4e...
6	00048cc3ae777c65dbb7d2a06...
7	00054e8431b9d7675808bcb8...
8	000576fe39319847cbb9d288c...
9	0005a1a1728c9d785b8e2b08...
10	0005f50442cb953dcd1d21e1f...
11	00061f2a7bc09da83e415a52d...

Insights

In this, we can see that the time to deliver each order and difference in estimated deliver for each order. We can easily get to know, is the order is taking more than estimated or less than estimated time to deliver the order.

5.2. Find out the top 5 states with the highest & lowest average freight value.

Query:

```
WITH highest AS (SELECT *, ROW_NUMBER() OVER() as rowh
FROM (SELECT c.customer_state, ROUND(AVG(oi.freight_value), 2) AS Highest_Average
FROM `target.order_items` oi JOIN `target.orders` o ON oi.order_id = o.order_id
JOIN `target.customers` c ON c.customer_id= o.customer_id
```

```

GROUP BY customer_state
ORDER BY Highest_Average DESC)
LIMIT 5),

lowest AS (SELECT *, ROW_NUMBER() OVER() as row1
FROM (SELECT c.customer_state, ROUND(AVG(oi.freight_value), 2) AS
Lower_Average
FROM `target.order_items` oi JOIN `target.orders` o ON oi.order_id
= o.order_id
JOIN `target.customers` c ON c.customer_id= o.customer_id
GROUP BY customer_state
ORDER BY Lower_Average ASC)
LIMIT 5)

SELECT h.customer_state AS Top5_Highest_Avg_State, h.Highest_Average, l.customer_state
AS Top5_lowest_Avg_State, l.Lower_Average
FROM highest h JOIN lowest l on h.rowh=l.rowl

```

Output

Untitled query ▶ RUN OPEN IN ⚙️ MORE 💾 SAVE ⬇️ DOWNLOAD 👤 SHARE 🕒 SCHEDULE ❌ Syntax error: Expected e

```

345 WITH highest AS (SELECT *, ROW_NUMBER() OVER() as rowh
346 FROM (SELECT c.customer_state, ROUND(AVG(oi.freight_value), 2) AS Highest_Average
347 FROM `target.order_items` oi JOIN `target.orders` o ON oi.order_id = o.order_id
348 JOIN `target.customers` c ON c.customer_id= o.customer_id
349 GROUP BY customer_state
350 ORDER BY Highest_Average DESC)
351 LIMIT 5),
352
353 lowest AS (SELECT *, ROW_NUMBER() OVER() as rowl
354 FROM (SELECT c.customer_state, ROUND(AVG(oi.freight_value), 2) AS Lower_Average
355 FROM `target.order_items` oi JOIN `target.orders` o ON oi.order_id = o.order_id
356 JOIN `target.customers` c ON c.customer_id= o.customer_id
357 GROUP BY customer_state
358 ORDER BY Lower_Average ASC)
359 LIMIT 5)
360
361 SELECT h.customer_state AS Top5_Highest_Avg_State, h.Highest_Average, l.customer_state AS Top5_lowest_Avg_State, l.Lower_Average

```

Query results 💾 SAVE RESULTS 🏠

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	Top5_Highest_Avg_State	Highest_Average	Top5_lowest_Avg_State	Lower_Average		
1	RR	42.98	SP	15.15		
2	PB	42.72	PR	20.53		
3	RO	41.07	MG	20.63		
4	AC	40.07	RJ	20.96		
5	PI	39.15	DF	21.04		

Results per page: 50 1 - 5 of 5

Insights

This output easily tells us that the top5 Highest Average that is RR, PB, RO, AC, PI has the highest average states and the top5 Lowest Average are SP, PR, MG, RJ, DF and one can easily check their average value accordingly.

5.3. Find out the top 5 states with the highest & lowest average delivery time.

Query:

```
WITH highest AS (SELECT *, ROW_NUMBER() OVER() as rowh
FROM (SELECT c.customer_state,
ROUND(AVG(DATE_DIFF(DATE(order_delivered_customer_date),
DATE(order_purchase_timestamp), DAY)), 2) AS Highest_Average
FROM `target.orders` o JOIN `target.customers` c ON c.customer_id=
o.customer_id
GROUP BY customer_state
ORDER BY Highest_Average DESC)
LIMIT 5),

lowest AS (SELECT *, ROW_NUMBER() OVER() as rowl
FROM (SELECT c.customer_state,
ROUND(AVG(DATE_DIFF(DATE(order_delivered_customer_date),
DATE(order_purchase_timestamp), DAY)), 2) AS Lower_Average
FROM `target.orders` o JOIN `target.customers` c ON c.customer_id=
o.customer_id
GROUP BY customer_state
ORDER BY Lower_Average ASC)
LIMIT 5)

SELECT h.customer_state AS Top5_Highest_Avg_State, h.Highest_Average, l.customer_state
AS Top5_lowest_Avg_State, l.Lower_Average
FROM highest h JOIN lowest l on h.rowh=l.rowl
```

Output

Untitled query RUN SAVE DOWNLOAD SHARE SCHEDULE OPEN IN MORE This query will process 8.32 MB when run

```
69
70 WITH highest AS (SELECT *, ROW_NUMBER() OVER() as rowh
71 FROM (SELECT c.customer_state, ROUND(AVG(DATE_DIFF(DATE(order_delivered_customer_date), DATE(order_purchase_timestamp), DAY)), 2) AS Highest_Average
72 FROM target.orders o JOIN target.customers c ON c.customer_id= o.customer_id
73 GROUP BY customer_state
74 ORDER BY Highest_Average DESC)
75 LIMIT 5),
76
77 lowest AS (SELECT *, ROW_NUMBER() OVER() as rowl
78 FROM (SELECT c.customer_state, ROUND(AVG(DATE_DIFF(DATE(order_delivered_customer_date), DATE(order_purchase_timestamp), DAY)), 2) AS Lower_Average
79 FROM target.orders o JOIN target.customers c ON c.customer_id= o.customer_id
80 GROUP BY customer_state
81 ORDER BY Lower_Average ASC)
82 LIMIT 5)
83 SELECT h.customer_state AS Top5_Highest_Avg_State, h.Highest_Average, l.customer_state AS Top5_lowest_Avg_State, l.Lower_Average
84 FROM highest h JOIN lowest l on h.rowh=l.rowl
```

Query results SAVE RESULTS OPEN IN

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	Top5_Highest_Avg_State	Highest_Average	Top5_lowest_Avg_State	Lower_Average		
1	RR	29.34	SP	8.7		
2	AP	27.18	PR	11.94		
3	AM	26.36	MG	11.95		
4	AL	24.5	DF	12.9		
5	PA	23.73	SC	14.91		

Results per page: 50 1 - 5 of 5 REFRESH

Insights

This output easily tells us that the top5 Highest Average Delivery Time that is RR, AP, AM, AL, PA has the highest average delivery states and the top5 Lowest Average delivery States are SP, PR, MG, DF, SC and one can easily check their average value accordingly.

5.4. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Query:

```

WITH highest AS (SELECT *
FROM (SELECT c.customer_state,
ROUND(AVG(DATE_DIFF(DATE(order_estimated_delivery_date),DATE(order_delivered_customer_
date), DAY)), 2) AS Highest_Average
FROM `target.orders` o JOIN `target.customers` c ON c.customer_id=
o.customer_id
GROUP BY customer_state
ORDER BY Highest_Average DESC)
LIMIT 5)

SELECT customer_state AS Top5_Fast_Del_State, Highest_Average
FROM highest

```

Output

69	WITH highest AS (SELECT *
70	FROM (SELECT c.customer_state, ROUND(AVG(DATE_DIFF(DATE(order_estimated_delivery_date),DATE(order_delivered_customer_date), DAY)), 2) AS Highest_Average
71	FROM `target.orders` o JOIN `target.customers` c ON c.customer_id=o.customer_id
72	GROUP BY customer_state
73	ORDER BY Highest_Average DESC)
74	LIMIT 5)
75	
76	SELECT customer_state AS Top5_Fast_Del_State, Highest_Average
77	FROM highest
78	
79	
80	
81	

Press Alt+F1 for Accessibility Options.

Query results [SAVE RESULTS](#) [OPEN IN](#)

JOB INFORMATION	RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	Top5_Fast_Del_State	Highest_Average			
1	AC	20.72			
2	RO	20.1			
3	AP	19.69			
4	AM	19.57			
5	RR	17.29			

Results per page: 50 1 - 5 of 5 |< < > >|

Insights

This output easily tells us that the top5 States where the order delivery is fast as compared to estimated date of delivery are AC, RO, AP, AM, RR

6. Analysis based on the payments:

6.1. Find the month on month no. of orders placed using different payment types.

Query:

```
SELECT
    FORMAT_DATETIME("%B", DATETIME(o.order_purchase_timestamp)) AS month,
    COUNT(o.order_id) AS different_payment_types
FROM `target.orders` o JOIN `target.payments` p on o.order_id =p.order_id
GROUP BY month
ORDER BY 2 DESC
```

Output

Untitled query

▶ RUN

💾 SAVE

⬇️ DOWNLOAD

👤 SHARE

🕒 SCHEDULE

🔗 OPEN IN

⚙️ MORE

85

SELECT

FORMAT_DATETIME("%B", DATETIME(o.order_purchase_timestamp)) AS month,

87

COUNT(o.order_id) AS different_payment_types

88

FROM `target.orders` o JOIN `target.payments` p on o.order_id = p.order_id

89

GROUP BY month

90

ORDER BY month

Press Alt+F1 for Accessibility Options.

Query results

💾 SAVE RESULTS

📈 OPEN IN

⬆️

JOB INFORMATION

RESULTS

CHART

JSON

EXECUTION DETAILS

EXECUTION GRAPH

Row	month	different_payment_types
1	August	11248
2	May	11079
3	July	10824
4	March	10349
5	June	9855
6	April	9780
7	February	8838
8	January	8413
9	November	7863
10	December	5896
11	October	5206
12	September	4535

Results per page:

50

1 - 12 of 12

⏪

⏴

⏵

⏩

Insights

From this, we can get to know that Month August has the highest count of the no of different payments done and September has the least count.

6.2. Find the no. of orders placed on the basis of the payment installments that have been paid.

Query:

```
SELECT
    payment_installments,
    COUNT(DISTINCT order_id) AS total_orders
FROM `target.payments`
GROUP BY payment_installments
ORDER BY payment_installments;
```

Output

Query results SAVE RESULTS OPEN IN

JOB INFORMATION	RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	payment_installment	total_orders			
1	0	2			
2	1	49060			
3	2	12389			
4	3	10443			
5	4	7088			
6	5	5234			
7	6	3916			
8	7	1623			
9	8	4253			
10	9	644			
11	10	5315			
12	11	23			
13	12	133			

[Load more](#)

Results per page: 50 1 - 24 of 24

Insights

From this we get to know that, payment installment 1 has the highest number of orders

Recommendations

- Increase Marketing & Promotions During Peak Seasons** – Since August has the highest number of orders and September the lowest, companies should focus on increasing promotional campaigns, discounts, and advertising in low-performing months to maintain a steady order volume.
- Optimize Inventory & Logistics Based on Demand** – A seasonal trend in ordering behavior suggests businesses should stock up in high-demand months and optimize supply chains accordingly.
- Encourage Repeat Purchases** – Since Brazilian customers mostly place orders in the afternoon, targeted email campaigns or limited-time offers during this period may boost sales.
- Improve Logistics in States with High Delivery Times** – States with the highest average delivery times (RR, AP, AM, AL, PA) may require better warehouse distribution, partnerships with faster carriers, or additional fulfillment centers.

5. **Analyze Delivery Time Gaps** – Orders in some regions are significantly delayed compared to estimated dates. Analyzing causes (e.g., weather conditions, transportation, or processing delays) can help improve estimated delivery accuracy.
6. **Optimize Freight Costs** – Freight charges are significantly higher in some regions. Cost analysis and route optimization can help reduce logistics expenses.
7. **Promote Installment-Based Payments** – Since most customers opt for a single installment, businesses can incentivize larger purchases by offering interest-free installments.
8. **Monitor Yearly Growth & Trends** – With a growing trend in e-commerce orders, ensuring scalability in infrastructure (warehousing, logistics, and payment processing) is crucial.